

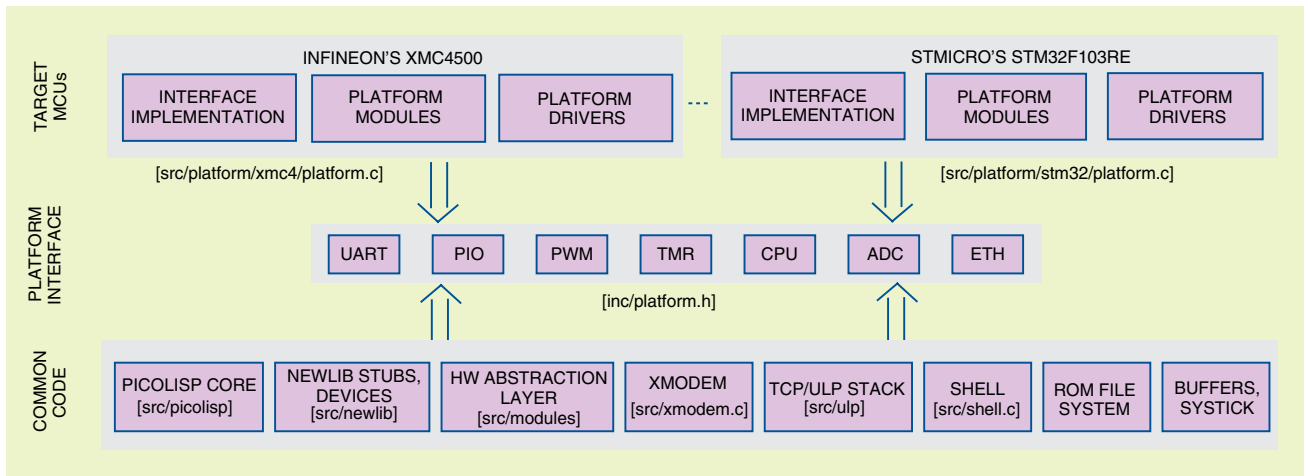


Fig. 2: Software system architecture for PicoLisp

SPI and timers. Each component has an identifier, which is a number that identifies that component in PicoLisp. Generally, numbers are assigned to components in their natural order. For example, Port A will have identifier value 0, Port B will have 1 and so on.

Similarly, second SPI interface (SPII) of the MCU will probably have an identifier value of 1. Pin 0 in Port 1 on Infineon XMC4500 will be called 'P1_0 in PicoLisp. Similarly, pin 27 in Port B on Atmel AT32UC3A0256 will be called 'PB_27 (notice the single opening quote).

Some issues

PicoLisp cannot be directly compiled for a 32-bit RISC machine. There are many issues to address before one can use PicoLisp REPL over UART or TCP/IP interface on the MCU. For instance, supporting programs such as memory allocators are required for PicoLisp to function correctly. We use Newlib C library and implement stubs of code for the memory allocator. We have to deal with issues like routing plain I/O over UART or TCP/IP interface of the MCU.

We also require support for a multimedia card memory interface to store PicoLisp programs. We can then load PicoLisp programs at runtime. This implies a requisite

for a file system. We also need to implement stubs of code for file I/O support over SPI protocol. For the file system, we use FatFs FAT file system module.

Once all supporting programs are in place, getting PicoLisp to run on the MCU is then fairly straightforward. On account of its small size, it can be easily embedded on an MCU in less than 256kB of flash. It can also be easily compiled for a given architecture with a toolchain like GNU gcc. Currently, a Python based build system called SCons is being used to compile the code base.

Bootting PicoLisp on the MCU

Given below is the sequence of events that occur after the MCU is powered up. The platform-initialisation code is executed. This program does very low-level platform setup, copies ROM contents to internal RAM, zeroes out BSS section, sets up the stack pointer and long jumps to the main function.

1. The main function calls the platform-specific initialisation function and returns a result that can be either a value indicating success or failure. If it fails, main instantly gets blocked. A debugger can then inspect the internals of the state machine.
2. The main function then initialises the rest of the system: ROM

file system, XMODEM and terminal support.

3. If files `/rom/autorun.l` or `/mmc/autorun.l` exist, these are executed. If one file is found before the other, it terminates further execution of the other file and the context jumps to the next step. If it returns after execution, or if files are not found, boot process continues with the next step.
4. If boot parameter is set to standard and the shell was compiled in the image, it is started. In the absence of the shell, standard PicoLisp server is started.

PicoLisp for MCUs

PicoLisp can be compiled to either support a user console over UART (the default and by far the most popular) or a console over TCP/IP. It can run on a wide variety of MCUs. Some practical aspects of using PicoLisp are listed below:

1. Code base is hardware independent. It proves to be extremely portable across different architectures.
2. Programs in PicoLisp are highly adaptable, field programmable and re-configurable for a variety of practical applications.
3. Programming the MCU follows a very natural iterative process because PicoLisp permits the user to develop programs in an